Applied Deep Learning

Image Classification, Convolution, Pooling, and Validation

Prepared by: Nhat Tam Huynh

Student ID: 101042941

Instructor: Zahra Atf

Course: MBAI 5310

Assignment 7

Introduction

This report answers ten short-answer questions based on the assigned readings on image classification, convolutional neural networks (CNNs), convolution, pooling, and model validation and error analysis. Each answer explains the underlying concept in plain language and includes an original, realistic example.

Question 1: What does image classification mean? Explain with one example.

Image classification is when a model looks at an image and decides which single category it belongs to from a fixed list of possible categories. The model takes in the picture as a grid of pixel numbers and outputs the label it thinks fits best. For example, if you gave a model a photo of a winter coat, it should look at the pixel patterns, such as the bulky shape and long sleeves, and predict the label "coat" rather than "sandal" or "bag." The model isn't actually seeing a coat the way a person does; it's matching learned numerical patterns to one of the categories it was trained on.

Question 2: Why is using a Dense Network not always a suitable choice for images?

A dense network treats an image as a flat list of numbers, so it loses the information about which pixels were actually next to each other. Since edges, shapes, and textures depend on nearby pixels working together, a dense layer must relearn the same pattern separately for every possible position in the image, which wastes a lot of weights and training data. For instance, if a dense network learns to recognize the curve of a shoe's sole in the bottom left of an image, it has no guarantee it will recognize that same curve if it shows up in the bottom right instead. It must learn that pattern all over again for the new location.

Aspect	Dense Network	Convolutional Network
Spatial awareness	None; flattens image, loses pixel layout	Preserves height and width relationships
Pattern reuse	Must relearn a pattern at every position	One filter reused everywhere (weight sharing)
Parameter count	Very high for large images	Much lower; filters are small and shared
Best suited for	Small, non-spatial, or tabular data	Images and other grid-structured data

Table 1. Dense networks vs. convolutional networks for image data.

Question 3: In a CNN, what is convolution, and what does a filter or kernel do?

Convolution is the operation where a small grid of numbers, called a filter or kernel, slides across the image and checks how well each small patch of the image matches the pattern the filter is looking for. At each position, the filter multiplies its values by the pixel values beneath it and sums the results into a single number. A filter built to detect horizontal lines, for example, would give a strong output when it passes over the brim of a hat where there's a clear horizontal boundary between light and dark pixels, but a weak output when it passes over a plain, smooth area of fabric.

Question 4: What is a feature map in a convolutional neural network?

A feature map is the output you get after a filter has finished sliding across the entire image. It's basically a smaller grid of numbers showing where in the image that particular pattern showed up strongly and where it didn't. If a filter is tuned to detect rounded corners, the resulting feature map will have high values in the regions of the image where rounded corners actually appear, like the toe area of a sneaker, and low values everywhere else, like a flat sleeve.

Question 5: Why is weight sharing used in CNNs? Explain.

Weight sharing means the same filter is reused at every position across the image instead of training a brand-new filter for each spot. This is useful because a meaningful pattern, like an edge or a curve, can mean the same thing no matter where it shows up in the picture, so there's no reason to relearn it over and over. Imagine a filter that detects the strap on a sandal. With weight sharing, that one filter can spot a strap whether it appears near the top of the image or off to the side, which saves the network from needing thousands of separate detectors and also makes training much faster.

Question 6: How does the ReLU activation function work? Write the ReLU output for the following values: [-3, 0, 2, 5]

ReLU is a simple rule applied to each number coming out of a layer: if the number is negative, it gets turned into zero, and if it's zero or positive, it stays exactly as it is. The idea is to keep only the "positive evidence" that a feature was detected, while throwing away negative signals that don't help indicate the feature's presence. This also adds nonlinearity, which lets the network represent more complicated relationships rather than just a straight-line transformation.

For the values [-3, 0, 2, 5], ReLU gives [0, 0, 2, 5].

Input value	-3	0	2	5
ReLU output	0	0	2	5

Table 2. ReLU output for the sample input values. Negative numbers become 0; non-negative numbers stay the same.

Question 7: What is max pooling, and why is it used in CNNs?

Max pooling shrinks a feature map by sliding a small window, often 2 by 2, across it and keeping only the highest value found in each window while discarding the rest. It's used because it reduces the size of the data the network has to process in later layers, which speeds up computation, and it also gives the model some flexibility if a feature shifts slightly. For example, if a button on a shirt appears one pixel to the left in one photo compared to another, max pooling can still capture that the button's strong signal was present somewhere in that small region, so the model isn't thrown off by tiny positional differences.

2 × 2 window values	Max pooling output
1, 3, 2, 0	3
4, 6, 5, 2	6
0, 1, 1, 0	1

Table 3. Max pooling keeps only the largest value from each small window, shrinking the feature map while keeping its strongest signals.

Question 8: In the Fashion-MNIST code, why are images reshaped from 28 × 28 to 28 × 28 × 1?

The Conv2D layer in Keras expects every image to come with a channel dimension in addition to height and width, because that's how it knows how many separate "color layers" to look at. A grayscale image like Fashion-MNIST technically only has one channel of brightness information, but a plain 28 × 28 array doesn't explicitly state that, so it has to be reshaped to 28 × 28 × 1 to tell the layer there is exactly one channel. If the dataset were color photos instead, the shape would be 28 × 28 × 3 for the red, green, and blue channels.

Question 9: What is the difference between training data, validation data, and test data?

Training data is the set of examples the model directly learns from, meaning its weights get updated based on these images and labels. Validation data is a separate set used while building and tuning the model, to check whether it's actually learning generalizable patterns rather than just memorizing the training examples, and it can guide decisions like when to stop training. Test data is kept completely separate and is only used once the model is finished, to get an honest, final estimate of how well it will perform on images it has truly never encountered, such as a fresh batch of clothing photos collected after the model was built.

	Training Data	Validation Data	Test Data
Purpose	Updates model weights	Checks generalization during development	Final, unbiased performance check
Used when	During every training step	Between epochs/training runs	Only after the model is finished
Example	Thousands of labeled clothing photos are used to fit the model	A held-out set used to tune layers or decide when to stop training	A fresh batch of photos is never touched until final evaluation

Table 4. How training, validation, and test data are each used in a typical deep learning workflow.

Question 10: What is overfitting? If training accuracy is high but validation accuracy is low, what does this indicate?

Overfitting happens when a model learns the training examples so specifically, including their quirks and noise, that it stops being able to handle new, unseen images well. Instead of learning general visual rules like “sneakers have a low, curved profile,” it might latch onto irrelevant details, such as a particular lighting style that happened to be common in its training photos.

If training accuracy is high but validation accuracy is low, this is a clear sign of overfitting. It means the model has gotten very good at recognizing the exact images it was trained on, but it hasn’t picked up patterns general enough to transfer to images it hasn’t seen before, so its performance drops noticeably once it’s tested on the validation set.

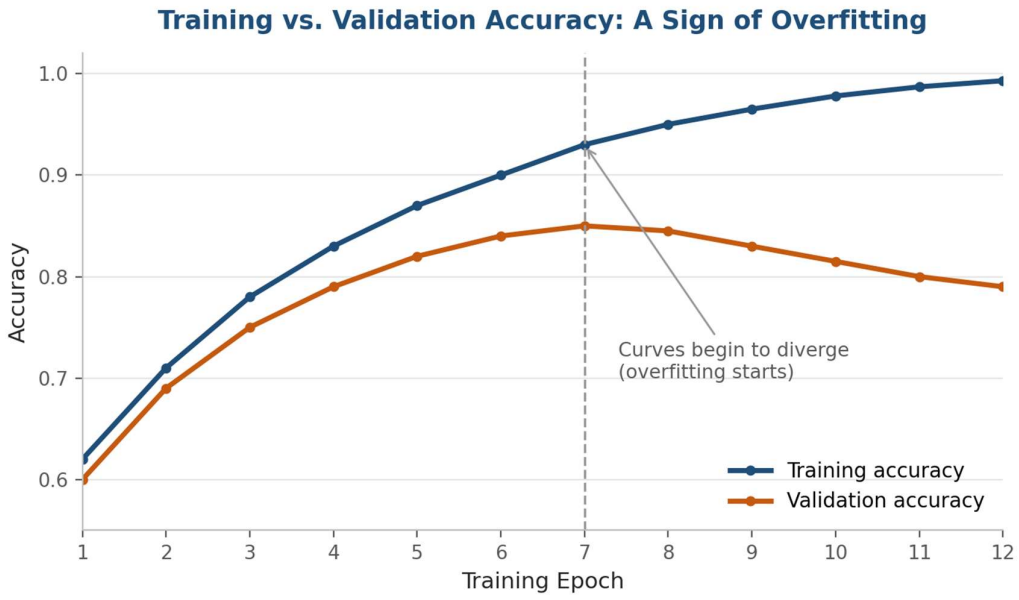

Figure 1. As training continues, training accuracy keeps climbing while validation accuracy levels off and then declines after epoch 7. The widening gap between the two curves is the classic signature of overfitting.